

Final Paper Submission:

Software Engineering in the Age of AI Seminar

“ICU Decision Support System”

Presenters:

Anzhelika Moon

Maksim Averbman

Adi Twizer

Tamar Kohan

16/08/2026



Index

1. Research question, objectives and importance.....	3
2. Engineering background and proposed method.....	4
2.1 Approach.....	4
2.2 Architecture.....	4
2.3 Algorithms.....	4
2.4 Tools.....	5
3. Implementation and evaluation.....	5
3.1 Data and cohort.....	5
3.2 Preprocessing at training time (export_model.py).....	6
3.3 Core components.....	7
3.4 Automated tests.....	8
3.5 Results and baseline comparison.....	9
4. Development process, challenges and lessons.....	10
4.1 Phases.....	10
4.2 Design decisions.....	10
4.3 Challenges and resolutions.....	11
4.4 Lessons.....	12
5. Conclusions and future work.....	13
5.1 Conclusions.....	13
5.2 Advantages.....	13
5.3 Limitations.....	13
5.4 Future work.....	14
6. User manual.....	15
6.1 Requirements.....	15
6.2 Installation.....	15
6.3 Running.....	15
6.4 Required inputs.....	16
6.5 Operating steps.....	16
6.6 Outputs.....	16
6.7 Repository structure.....	17
6.8 System Evaluation & Testing Strategy.....	17
6.8.1. Automated Testing Suite (PyTest).....	17
6.8.2. Manual End-to-End User Journey Testing.....	18
6.8.3 Testing Summary & Evaluation Verdict.....	20
7. First paper.....	21
7.1 Proposed approach, main contribution, and key findings.....	21
7.2 Contribution to the project and how it is practical to application.....	21
8. Second paper.....	21
8.1 Proposed approach, main contribution, and key findings.....	22
8.2 Contribution to the project and how it is practical to application.....	22

1. Research question, objectives and importance

The problem: In an ICU the data needed to spot a deteriorating patient already exists ,vitals are charted continuously and labs return within hours. The problem is attention, not measurement. A nurse covering several beds sees numbers that look normal one at a time: the *combination* that signals shock is only visible if someone reads heart rate, blood pressure and lactate together. So deterioration is often noticed late, and late in an ICU means organ damage.

Research question: How can early clinical indicators from the first 12 hours of ICU admission be used to identify patients at risk of deterioration, in order to support medical staff in decision-making?

Why 12 hours: This is the period when the clinical picture is least clear, staff know least about the patient, and intervention can still change the outcome. It is also a hard data boundary: anything measured after hour 12 is excluded, so the model cannot use information a clinician would not have had.

Objectives:

1. Produce a risk estimate per patient using first-12-hour data only.
2. Explain each estimate for that specific patient, not just globally.
3. Turn the ranked risk into a workable clinical artefact: a priority queue, an alert lifecycle, recommended actions that can be acknowledged, and a statement of what data is missing.
4. Serve two users, not one ,the clinician at the bedside, and the charge nurse or unit manager who needs a view of the department as a whole.
5. Deliver it as maintainable software: persistent storage, authentication and roles, automated tests, and clear separation between UI, decision logic, features and model.

Contribution: Not a better mortality model ,those exist. The contribution is showing that a modest, honest model becomes clinically useful only when it is wrapped in decision-support engineering: prioritisation, per-patient explanation, an alert workflow with an audit trail, re-assessment timing, and missing-data awareness. The model is one component, not the system.

Stated limitation: MIMIC-III has no "deteriorated" label, so the model learns HOSPITAL_EXPIRE_FLAG (in-hospital mortality) as a proxy. Mortality is a later and coarser outcome than deterioration.

2. Engineering background and proposed method

2.1 Approach

Two stages:

Stage 1: a supervised classifier maps early indicators to a risk probability.

Stage 2 (where most of the engineering is): converts that probability into decision support, risk bands, per-patient explanation, rule-based alerts, prioritisation, an alert state machine with an audit trail, re-assessment intervals, what-if simulation, department-level analytics and missing-data recommendations.

2.2 Architecture

Layered modular desktop application, one Python process, backed by a local SQLite database:

Clinician → CustomTkinter GUI (app.py), login, roles, screens → Patient management + decision-support logic → SQLite (icu_system.db): patients, users [persistence layer]
→ Inference module (backend/app.py, imported directly) → compute_features() → scaler.transform() → model.predict_proba() → get_risk_drivers() + generate_alerts()
→ risk score, band, drivers, alerts back to the GUI

The GUI imports the inference module directly (from backend.app import compute_features, generate_alerts, get_risk_drivers, model, scaler, FEATURES, THRESHOLD), so every prediction is a function call in memory: no HTTP, no serialisation, no second process. Model artifacts load once at import. call_model() is the single point where the GUI touches the model.

Note on Flask: backend/app.py still constructs a Flask object and defines a /predict route, and frontend/ still holds the templates and stylesheet for it. This is a superseded interface: it works only if run by hand and the desktop application never touches it (commit: "Remove Flask dependency: call ML model directly from GUI"). The repository contains a REST interface; the product does not use one.

2.3 Algorithms

- **Model:** LGBMClassifier (gradient-boosted trees), is_unbalance=True, n_estimators=300, max_depth=6, learning_rate=0.05, random_state=42. Chosen over logistic regression, random forest and XGBoost based on the experiment log (3.4).

- **Class imbalance:** mortality is roughly a 1-in-10 event, so accuracy is meaningless (predicting "survives" for everyone scores ~90%). Handled by loss reweighting plus moving the decision threshold.
- **Threshold:** chosen by Youden's J (max TPR – FPR) rather than left at 0.5, giving **0.4361**. A missed deteriorating patient costs more than an unnecessary check, so the threshold is biased toward sensitivity.
- **Risk bands:** CRITICAL ≥ 0.60 · HIGH ≥ 0.4361 · MODERATE ≥ 0.25 · LOW otherwise.
- **Per-patient explanation:** `get_risk_drivers()` uses LightGBM's built-in `pred_contrib=True` attribution. Each feature gets a signed log-odds contribution; drivers are ranked by magnitude and shown with a clinician-readable label, the patient's own value with units, and direction (▲ red = raised risk, ▼ green = lowered). No external SHAP dependency.
- **Rule-based alerts:** `generate_alerts()` applies explicit clinical thresholds independently of the model, tachycardia (HR > 100), bradycardia (HR < 50), SysBP_min < 90, shock index > 0.9 (warning > 0.7), MAP < 65, age > 75, HR range > 40, pulse pressure < 25, lactate > 4, creatinine > 2, tagged critical / warning / info. Deliberate redundancy: legible and auditable even when the model is not.
- **Re-assessment intervals.** Each risk band carries a review interval, CRITICAL 15 min, HIGH 30, MODERATE 60, LOW 240. `get_reassessment_info()` compares it against `last_reviewed_at` and reports either time remaining or how long the patient is overdue; CRITICAL patients that are overdue or never reviewed raise a banner across the top of the dashboard. This turns a static score into something with a clock on it.

2.4 Tools

Python 3.12 (3.14 is incompatible with the ML stack) · pandas / NumPy / Google Colab for data engineering · LightGBM and scikit-learn for modelling (XGBoost and SMOTE for comparison only) · joblib + JSON for artifacts · SQLite for persistence · CustomTkinter + Pillow for the GUI · pytest for automated tests · Jupyter and CSV logs for experiments · Git/GitHub · MIMIC-III v1.4.

3. Implementation and evaluation

3.1 Data and cohort

Source: MIMIC-III v1.4 (PATIENTS, ADMISSIONS, ICUSTAYS, D_ITEMS, CHARTEVENTS, LABEVENTS). ICUSTAYS joined to PATIENTS and ADMISSIONS gives **61,532 ICU stays**.

Vitals: CHARTEVENTS (~330M rows) is too large for memory, so it was streamed in 67 chunks of 5M rows, reading only four columns. Each chunk was filtered to the target ITEMIDs, joined to INTIME and restricted to the 12-hour window. ITEMIDs cover both MIMIC source systems, CareVue and MetaVision, using only one set silently loses about half the

measurements (HR: 211, 220045; SysBP: 51, 455, 220050, 220179, 224167, 227243; DiasBP: 8368, 8440, 8441, 220051, 220180, 224643, 227242).

python

```
chunk['HOURS_SINCE_ADMISSION'] = (chunk['CHARTTIME'] -
chunk['INTIME']).dt.total_seconds() / 3600
first_12_hours = chunk[(chunk['HOURS_SINCE_ADMISSION'] >= 0) &
(chunk['HOURS_SINCE_ADMISSION'] <= 12)]
```

Result: **2,660,721 readings** inside the window, aggregated per stay as mean/min/max into 9 columns and merged back with a left join (patients with no readings are kept as NaN, not dropped).

Labs: LABEVENTS streamed in 6 chunks for lactate (50813) and creatinine (50912). LABEVENTS keys on HADM_ID rather than ICUSTAY_ID, so a bridging lookup was built before applying the same 12-hour filter. **100,034 results**, aggregated as max per stay, for a risk marker the worst value carries the signal.

Age: Computed as INTIME.year – DOB.year, not by datetime subtraction: MIMIC shifts DOB for patients over 89 by ~300 years, which overflows pandas datetimes. Ages above 150 map to 90.

Filters: AGE >= 16 removes neonatal patients. Output: **53,424 adult ICU stays** in COMPLETE_ICU_RISK_DATASET.csv.

3.2 Preprocessing at training time (export_model.py)

The Colab output was structurally clean but not physiologically clean, so training adds a second stage:

- **Outlier removal:** Impossible values (zeros meaning "no reading", negatives, SysBP = 127105) are set to NaN using a valid_ranges dictionary: HR 20–300, SysBP 30–300, DiasBP 10–200, creatinine 0.1–50, lactate 0.1–40.
- **Missingness filter:** Stays with more than 5 missing values across the 11 measurement columns are dropped, as is AGE < 3.
- **Missingness as signal:** Binary indicators are created *before* imputation. Whether a clinician ordered a lactate reflects concern about the patient, and keeping that improved the model (3.4).
- **Imputation:** Median fill, with the medians saved into model_meta.json so the GUI imputes identically.
- **35 features:** 9 vitals (mean/min/max), AGE, GENDER_M; ranges (BP_range, HR_range, SBP_range); composites (shock_index, pulse_pressure, MAP); interactions (age_x_hr, age_x_sbp); threshold flags (shock_index_high, age_over_75, map_low, tachycardia, hypotension); admission timing (admit_hour, admit_dayofweek, is_night, is_weekend); 5 missingness indicators; 2 labs.

- **Split and scaling:** 80/20 stratified split, random_state=42; StandardScaler fit on train only.
- **Artifact contract:** Training writes model.joblib, scaler.joblib and model_meta.json (feature order, threshold, metrics, ranges, medians). model_meta.json is the single source of truth between training and serving ,regenerated, never hand-edited.

3.3 Core components

Component	Responsibility
compute_features()	Turns ~10 raw inputs into the 35 model features, in FEATURES order
call_model()	Single entry point: features → scaling → probability → band → drivers → alerts
get_risk_drivers()	Top-N signed per-patient drivers via LightGBM pred_contrib
generate_alerts()	Independent rule-based clinical alerts
get_reassessment_info()	Band-based review interval vs. last_reviewed_at ,due, overdue, or pending
get_missing_model_fields()	Guards every screen: no score is shown when core physiology is absent
SQLite layer	init_database, save_patient_to_database, save_patients_bulk, load_patients_from_database, authenticate_user, add_user_to_database, delete_user_from_database, get_all_users
Login	Username/password against the users table, SHA-256 hashed; role returned with the session
Dashboard	All patients with score, band, room and review status; overdue-CRITICAL banner
Priority Queue	Patients ranked by risk with attention level
Patient Details	Demographics, vitals, labs, room, audit fields; vitals editing with validation
Risk Explanation	"Why this risk score?" ,drivers with value, direction and magnitude
Recommended Actions	Band-specific steps as checkboxes; acknowledging them writes to the record
Alerts Workflow	New → Reviewed → In Progress → Resolved, paginated, each transition stamped
Department KPI	Unit-level risk stratification, clinical averages, CSV export
What-if Analysis	Clinician edits a value and sees the score recompute

Trend Monitoring	Directional change over time
Missing Data	Flags absent labs and recommends ordering them
Admin Console	Add and delete users (Admin role only)
Add Patient / Upload CSV	Manual entry and bulk import from MIMIC-style columns

Persistence: `icu_system.db` holds two tables. `patients` is keyed on `icustay_id` and written with `INSERT OR REPLACE`, so re-importing the same ICU stay updates the row instead of duplicating it, one of the properties covered by the test suite. Bulk CSV import uses `executemany` rather than a per-row loop. `init_database()` also performs a small forward migration: it reads `PRAGMA table_info(patients)` and adds `room_number` with `ALTER TABLE` if an older database is missing it, so existing installations survive a schema change without losing data.

Users and roles: The `users` table stores SHA-256 password hashes and one of three roles: Doctor, Nurse, Admin. Roles are not decorative, the Admin Console is only reachable as Admin, and room reassignment and clinical overrides are refused to anyone who is not a Doctor. Three demo accounts are seeded on first run with `INSERT OR IGNORE`, so restarting never overwrites real users.

Audit trail: Every alert transition and acknowledged protocol records `last_reviewed_at` and `reviewed_by`, and appends a timestamped line to the patient's clinical note ([`PROTOCOL EXECUTED: ... completed at HH:MM by <staff>`]). Acknowledging actions also moves the alert to *In Progress*. The result is that "who looked at this patient, when, and what did they do" is answerable, which is the difference between a display and a clinical record.

Department KPI: The first screen written for a different user. Every other screen answers "what about *this patient*"; this one answers "how is *this unit* doing", patient counts and percentages per risk band with the expected response for each, department averages for age, heart rate and blood pressure across patients with usable vitals, a refresh, and an export. The CSV carries identifiers, room, risk band and score, alert status, vitals, labs, physician note, and the review audit fields, so a shift handover or an external review has the full picture in one file.

Critical invariant: `compute_features()` in the inference module must mirror `export_model.py` exactly, same formulas, same order, enforced by `X = pd.DataFrame([feature_dict])[FEATURES]`. A mismatch here is the most likely cause of wrong predictions and produces no error.

3.4 Automated tests

`pytest`, under `tests/`. `conftest.py` provides a fixture that redirects `DATABASE_PATH` to a `tmp_path` SQLite file with `monkeypatch` and clears the in-memory patient list, so tests never touch the real database and clean up automatically. `test_system.py` covers:

-

- **Persistence:** a saved patient reloads with identical fields; data survives a simulated restart; saving the same icustay_id twice updates rather than duplicates.
- **Authentication:** hashing is deterministic, different passwords hash differently, a valid login returns the right role, a wrong password returns None.
- **User management:** the Admin role is preserved on round-trip, and a duplicate username raises sqlite3.IntegrityError.
- **Model:** two identical patients produce an identical score and band, determinism, which is what catches an accidental change to feature order or scaling.

3.5 Results and baseline comparison

Iteration 1: does feature engineering help? (logistic regression, features added cumulatively)

Exp	Features	Model	AUC	Recall	Macro-F1
E001	13 raw (baseline)	LogReg	0.766	0.106	0.564
E004	+ composites + missingness flags	LogReg	0.776	0.120	0.575
E006	+age interactions+ admission timing	LogReg	0.782	0.123	0.576
E007	all 32	RandomForest	0.807	0.150	0.598
E008	all 32	GradientBoosting	0.805	0.173	0.608

Two findings: engineered clinical features gave a real but modest lift (+0.016 AUC), nonlinear models gave more (+0.041), but recall stayed catastrophic. At ~15% sensitivity the system misses 6 of every 7 deteriorating patients. AUC 0.807 concealed a clinically useless model.

Iteration 2: imbalance and threshold (1,274 positives in the test set):

Exp	Model	Thr	Sens	Spec	PPV	Macro-F1	AUC	Detected
E01	LR baseline	0.50	0.081	0.995	0.696	0.541	0.735	103/1274
E02	LR balanced	0.50	0.646	0.698	0.229	0.569	0.737	823/1274
E04	XGB balanced	0.50	0.579	0.767	0.257	0.598	0.749	737/1274
E05	LGBM balanced	0.50	0.602	0.751	0.252	0.593	0.754	767/1274
E06	XGB + SMOTE	0.50	0.182	0.976	0.514	0.602	0.737	232/1274
E07	RF + SMOTE	0.50	0.418	0.848	0.276	0.606	0.732	532/1274
E08	LGBM balanced, tuned	0.44	0.711	0.658	0.225	0.558	0.754	906/1274

Discussion:

- *Class weighting mattered most.* E01 → E02 raised detection from 103 to 823 patients with almost no AUC change (0.735 → 0.737). AUC was nearly blind to the change that mattered clinically.
- *SMOTE lost to simple reweighting.* Synthesising minority patients in a 35-dimensional space of correlated physiological values appears to produce implausible patients rather than signal.
- *Macro-F1 and sensitivity disagree.* E07 has the best macro-F1 (0.606) but detects 532 patients; E08 has a worse macro-F1 (0.558) and detects 906. E08 was chosen on the clinical argument that a missed deterioration outweighs a false alarm, a documented trade-off, not a metric-maximisation result.
- *Precision is the honest weakness.* At PPV 0.225, roughly three of four flagged patients will not die in hospital. This is the accepted cost of 71% sensitivity, and exactly why the system is built as prioritisation and explanation rather than an autonomous alarm: a clinician can dismiss a false positive in seconds when they can see the drivers, but an opaque siren at this precision would be ignored within a week.
- *Leakage was checked explicitly*, not assumed. The notebook's leakage audit records that no feature correlates above 0.9 with the target (max 0.287), that accuracy is not suspiciously high, that SMOTE touched only training data, and flags the one real problem: the threshold was selected on the test set.

Production model: After the outlier cleaning and cohort filtering in 3.2: **AUC 0.7856, threshold 0.4361, sensitivity 0.73, macro-F1 0.58**, better than E08's 0.754 AUC, attributable to the physiological-range cleaning that the iteration-2 notebook did not apply.

4. Development process, challenges and lessons

4.1 Phases

Research question and dataset selection → scope calibration → data engineering in Colab → modelling iteration 1 (baseline + features) → modelling iteration 2 (imbalance + threshold) → export and serving contract → GUI construction → repository consolidation → architecture simplification (removing the Flask hop) → authentication and roles → SQLite persistence and schema migration → decision-support depth (re-assessment intervals, acknowledgeable actions, audit trail) → department-level analytics and export → automated tests.

Early framings of the question were either pure software specification or unbounded deep learning; it was narrowed to something answerable from the data whose answer implies a software need. LSTM / real-time streaming / heavy deep learning were explicitly rejected as infeasible for a semester, and pure rule-based scoring as insufficiently data-driven.

4.2 Design decisions

- **Mortality as proxy label:** No deterioration label exists in MIMIC-III; the limitation was documented rather than abandoning the question.
- **12-hour window enforced at extraction, not by filtering later:** Post-hour-12 data never entered the pipeline, closing the door to leakage through a forgotten join.
- **Missingness treated as a feature:** since ordering a lactate encodes clinician concern.
- **Threshold moved off 0.5:** The default is a mathematical convention, not a clinical one.
- **Rule-based alerts kept alongside the model:** legible, auditable, independent of model failure.
- **Flask layer collapsed:** Two processes, a port and a serialisation boundary bought nothing for a single-user desktop tool. The endpoint stays in the repo as a migration path.
- **Persistence keyed on icustay_id, not a surrogate row id:** The ICU stay is the natural unit of the analysis and of the dataset, so making it the primary key means re-importing a CSV corrects records instead of multiplying them. Idempotent import was cheaper to design in than to retrofit.
- **Schema migration instead of "delete the database and start over":** Adding room_number to an existing install via PRAGMA table_info + ALTER TABLE costs a few lines and preserves whatever clinical notes are already stored.
- **Recommended actions made acknowledgeable:** A list of suggestions the clinician cannot respond to is a leaflet. Once acknowledgement is recorded ,who, when, which actions ,the same screen becomes part of the clinical record and closes the alert loop.
- **Re-assessment tied to the risk band:** A CRITICAL patient reviewed 40 minutes ago is a different situation from a CRITICAL patient reviewed 2 minutes ago, and the score alone cannot express that.
- **A screen for the unit, not the patient:** The KPI page was added deliberately for a second user ,a charge nurse or unit manager ,because prioritisation only makes sense relative to the whole department.
- **Tests written against behaviour, not implementation:** "The same ICU stay must not duplicate" and "two identical patients must score identically" are properties that stay true through refactoring.

4.3 Challenges and resolutions

Challenge	Resolution
CHARTEVENTS (~330M rows) exceeds memory	Chunked streaming with usecols ,filter early, keep only survivors
Vitals split across CareVue and MetaVision	Resolved all variants against D_ITEMS: 15 ITEMIDs for 3 vitals
LABEVENTS keys on HADM_ID, not ICUSTAY_ID	Bridging lookup table before applying the time window

Age computation overflowed pandas datetimes	Year arithmetic; mapped the 300-year de-identification artifact to 90
Colab lost the dataset between sessions	Persisted intermediates to Drive with a reload cell
Impossible values survived the Colab pipeline	Second cleaning stage via <code>valid_ranges</code> before imputation; worth ~0.03 AUC
Recall ~15% despite AUC 0.807	Diagnosed as imbalance, not model capacity; fixed with <code>is_unbalance</code> + threshold tuning
SMOTE degraded performance	Kept in the log as a negative result
Python 3.14 incompatible with the ML stack	Pinned to 3.12 via <code>py -3.12 -m venv</code>
<code>show_title</code> collided with Tk's built-in title	<code>self.title(...)</code> for the window, <code>self.show_title(...)</code> for headings
Two disconnected projects (Git repo + PyCharm GUI)	Consolidated into one repository
Re-importing a CSV created duplicate patients	<code>icustay_id</code> as primary key with <code>INSERT OR REPLACE</code> ; locked in by a regression test
Adding room numbers would have broken existing databases	Runtime schema check and <code>ALTER TABLE</code> on startup
Editing vitals left a stale cached score on the patient	Explicitly clear <code>_risk_score</code> , <code>_risk_level</code> and <code>model_result</code> before recomputing
Tests would have written to the live <code>icu_system.db</code>	monkeypatch of <code>DATABASE_PATH</code> onto a pytest <code>tmp_path</code> fixture
Patients with missing core vitals produced meaningless scores	<code>get_missing_model_fields()</code> gate, the UI states what is missing instead of showing a number

4.4 Lessons

- **A single metric will mislead you:** AUC 0.807 with 15% recall looks good and helps nobody. Reporting sensitivity, specificity, PPV, macro-F1 and raw detected count side by side fixed this, "906 of 1,274 detected" communicated more in one glance than any AUC.
- **Negative results are results:** Keeping the SMOTE failures made the final choice defensible instead of arbitrary.
- **Baseline first.** Because E001 existed, every later improvement was measurable rather than asserted.
- **Duplicated logic across a boundary is a latent bug:** Feature engineering lives in both training and inference and must match exactly; the honest fix is a shared module.

- **Architecture should match deployment reality:** The Flask layer was good engineering for a problem the project did not have; removing it was harder than adding it.
- **Data engineering was most of the work:** Extraction, ITEMID reconciliation, chunking and de-identification artifacts took far more effort than model training.
- **State is where the real bugs live:** Almost every late defect came from cached scores, duplicated rows or schema drift ,not from the model. Persistence forced questions the in-memory prototype never had to answer.
- **Tests pay for themselves at refactor time, not at write time:** The determinism and no-duplicate tests are cheap to write and would have caught the two most damaging silent failures in the system.

5. Conclusions and future work

5.1 Conclusions

- Early ICU indicators do carry a usable risk signal: $AUC \approx 0.79$ from heart rate, blood pressure, age and two labs in a 12-hour window. Real but moderate ,enough to prioritise attention, not to decide anything.
- Handling imbalance and choosing the threshold mattered more than model architecture. The gap between logistic regression and LightGBM was smaller than the gap between threshold 0.5 and 0.44 on the same model.
- Engineered clinical features helped modestly and consistently; missingness indicators helped ,absence of data is data.
- Most importantly, usefulness came from the decision-support layer, not the classifier. At PPV 0.225 a bare alarm would be ignored. The same score, with per-patient drivers, a priority ranking, an alert lifecycle with an audit trail, a re-assessment clock, and an explicit statement of what is missing, is something a clinician can act on ,and something a unit manager can review afterwards.

5.2 Advantages

12-hour constraint enforced at extraction, so the model cannot see the future • per-patient rather than global explanations • rule-based alerts independent of the model • threshold and medians live in model_meta.json, so the model is swappable without touching the UI • every modelling claim traces to a logged experiment with a baseline • single process, no "is the backend up?" failure mode • persistent storage with idempotent import and forward schema migration • authentication with hashed passwords and enforced role permissions • a full audit trail of who reviewed what and when • department-level view and export for handover • automated tests over persistence, authentication and model determinism • the system states what it doesn't know rather than scoring on absent data.

5.3 Limitations

- **Proxy label:** predicts in-hospital mortality, not deterioration.

- **Low precision:** $PPV \approx 0.225$; roughly three of four flags are false positives ,an accepted trade, but a real alert-fatigue risk.
- **Threshold tuned on the test set:** so the reported operating point is mildly optimistic; nested CV is needed.
- **68% lactate missingness:** filled with the cohort median; the missing flag mitigates but does not solve this.
- **Narrow feature set:** no temperature, respiratory rate, SpO_2 , GCS, urine output, diagnoses, medications or comorbidities.
- **Static snapshot, not a trajectory:** mean/min/max lose the direction of change, which is much of what clinicians read.
- **Single-centre, retrospective:** one US academic hospital, 2001-2012, no external validation.
- **Authentication is demonstration-grade:** appropriate for a prototype but not for real patient data.
- **Re-assessment intervals are fixed constants:** not configurable per unit or per role.
- **Test coverage is narrow.** Persistence, authentication and determinism are covered; `compute_features()` itself has no golden-value test asserting training/inference parity, which is the highest-risk untested surface.
- **The GUI is still a single large file:** and feature-engineering logic remains duplicated across training and inference.
- **Vestigial Flask and frontend/code:** remain in the repository, unused by the product.
- **Rejected features retained:** `is_night`, `is_weekend`, `admit_hour`, `admit_dayofweek` had minimal impact but stayed in the feature list.
- **Not validated for clinical use:** research and educational purposes only.

5.4 Future work

Data and modelling: Add respiratory rate, SpO_2 , temperature, GCS and urine output. Replace mean/min/max with trajectory features or a sequence model so direction of change becomes visible. Select the threshold by nested cross-validation to remove test-set leakage. Validate externally on eICU or MIMIC-IV. Model a true deterioration event ,vasopressor initiation, unplanned intubation, rapid-response call ,instead of mortality.

Software: Extract feature engineering into one shared module imported by both training and inference, with a golden-value parity test asserting that the two produce identical feature vectors. Split the GUI into per-screen modules. Remove or complete the Flask interface rather than leaving it ambiguous. Extend the test suite to the alert state machine and the risk-band boundaries. Harden authentication if the system were ever to move beyond demonstration use.

Clinical and human factors: Alert-fatigue mitigation: suppression rules and per-unit configurable thresholds and review intervals. Role-differentiated presentation beyond permissions ,a nurse and a senior physician should not necessarily see the same screen. Escalation pathways when a CRITICAL patient stays overdue. Calibration display so clinicians can judge the score's reliability. Prospective usability evaluation with real ICU staff.

6. User manual

6.1 Requirements

Windows, macOS or Linux with a graphical desktop. **Python 3.12**, not 3.14, which several ML dependencies do not yet support (py -0 on Windows lists installed versions). Dependencies: flask 3.1.3, numpy 2.1.1, pandas 2.2.3, scikit-learn 1.7.1, joblib 1.5.1, lightgbm 4.6.0, plus customtkinter, pillow and pytest. A few hundred MB of disk; the dataset CSV is needed only for retraining. The database file is created automatically on the first run.

6.2 Installation

bash

```
git clone https://github.com/MaksimAverman/SE-work-seminar.git
cd SE-work-seminar
```

```
py -3.12 -m venv .venv      # Windows
# python3.12 -m venv .venv   # macOS / Linux
```

```
.venv\Scripts\activate      # Windows
# source .venv/bin/activate  # macOS / Linux
```

```
pip install -r backend/requirements.txt
```

Symptom	Cause and fix
Could not open requirements file	You are not in the repository root ,cd into it, the path is relative
'pip' is not recognized	venv not activated; use py -3.12 -m pip install -r backend/requirements.txt
LightGBM build failure	You are on Python 3.14 ,recreate the venv with 3.12

6.3 Running

bash

```
python app.py
```

A login window opens; icu_system.db is created on first launch and seeded with demo accounts. Do not start a separate backend ,the model runs inside this process.

Username	Password	Role	Can
doctor1	doctor123	Doctor	Everything clinical, including room assignment and clinical overrides
nurse1	nurse123	Nurse	View, review alerts, acknowledge actions
admin1	admin123	Admin	Admin Console: add and delete users

These are demonstration credentials seeded in code. Replace them before any real use.

Tests: pytest from the repository root. Tests run against a temporary database and never touch icu_system.db.

6.4 Required inputs

Mandatory: age (years), gender (M/F), heart rate mean/min/max (bpm), systolic BP mean/min/max (mmHg), diastolic BP mean/min/max (mmHg), all from the first 12 hours. $\text{Min} \leq \text{mean} \leq \text{max}$ is enforced on entry and on edit.

Optional (blank is handled: imputed with the cohort median and flagged internally as missing): max creatinine (mg/dL), max lactate (mmol/L), admission hour (0–23), day of week (0–6), room number (digits 1–999, stored as ICU-*nnn*).

Bulk CSV import expects MIMIC-style columns: SUBJECT_ID, HADM_ID, ICUSTAY_ID, INTIME, GENDER, AGE, HeartRate_/SysBP_/DiasBP_ mean/min/max, and optionally Creatinine_max, Lactate_max. Missing required columns are reported by name and the import is refused; invalid rows are skipped and counted. Re-importing the same ICU stay updates the existing record rather than creating a duplicate. HOSPITAL_EXPIRE_FLAG, if present, is stored as metadata only and never passed to the model, it is the outcome label, and feeding it in would be target leakage.

6.5 Operating steps

Automatic: database initialised and migrated if needed; saved patients reloaded; artifacts loaded; 35 features derived; blanks imputed and flagged; scaling and inference; risk banding; driver attribution; alert generation; re-assessment status computed per band; overdue-CRITICAL banner raised; admit_hour / admit_dayofweek derived from INTIME on import; every review and acknowledgement timestamped and written to the database.

Manual: log in → add patients manually or by CSV → review the Dashboard, starting with any overdue-CRITICAL banner → work the Priority Queue in order → open Risk Explanation for a specific patient → tick and acknowledge Recommended Actions (Doctors can add an override reason) → advance alerts (New → Reviewed → In Progress → Resolved) → use What-if Analysis → check Missing Data for labs worth ordering → open Department KPI for the unit-level view and export → switch theme or user from the sidebar as needed.

6.6 Outputs

risk_score (0–1) · risk_percent · risk_level (LOW / MODERATE / HIGH / CRITICAL, colour-coded) · is_at_risk (score vs threshold) · risk_drivers (ranked factors with value, direction ▲/▼ and impact) · clinical_alerts (rule-based, with severity) · computed_indicators (shock index, MAP, pulse pressure, ranges) · priority ranking · recommended actions and their acknowledgement state · alert status with reviewer and timestamp · re-assessment status · department KPI summary · exported department CSV.

All outputs are decision support. They are not a diagnosis or a clinical instruction.

6.7 Repository structure

```
SE-work-seminar/
├── app.py                # CustomTkinter desktop application (entry point)
├── icu_system.db          # SQLite database, created on first run (gitignored)
├── backend/
│   ├── app.py            # inference module (+ vestigial, unused Flask /predict route)
│   ├── export_model.py    # training pipeline → writes the artifacts below
│   ├── requirements.txt
│   └── models/
│       ├── model.joblib    # trained LightGBM classifier
│       ├── scaler.joblib   # StandardScaler fitted on the training split
│       └── model_meta.json # feature order, threshold, metrics, ranges, medians
├── tests/
│   ├── conftest.py        # temporary-database fixture
│   └── test_system.py     # persistence, authentication, model determinism
├── frontend/              # templates and stylesheet for the unused Flask route
├── data/COMPLETE_ICU_RISK_DATASET.csv # 53,424 adult ICU stays (MIMIC-III derived)
├── notebooks/             # Colab extraction + modelling iterations 1 and 2
├── experiments/           # experiment_log.csv, experiment_log_final.csv
├── images/
├── docs/MODEL_INTEGRATION.md
└── README.md
```

Retraining: `cd backend && python export_model.py`, then restart the application. Threshold, metrics and medians propagate automatically because they are read from `model_meta.json` at startup. Never hand-edit that file.

6.8 System Evaluation & Testing Strategy

To validate the reliability, clinical safety, data integrity, and user experience of the ICU Clinical Decision Support System, a dual evaluation strategy combining Automated Unit & Integration Testing (via PyTest) and Manual End-to-End User Journey Testing was conducted.

6.8.1. Automated Testing Suite (PyTest)

Automated test suites were designed using pytest to verify core backend components, database transactions, cryptographic integrity, and ML prediction deterministic behaviors. A dynamic fixture (`test_database`) was leveraged to isolate tests within an in-memory/temporary SQLite instance without mutating production records.

- Database Persistence & State Integrity: Validated CRUD cycles across patient admission records, persistence of vital updates, room reassignments, clinical override notes, and duplicate admission rejection.

- Security & Authentication Protocol: Verified SHA-256 password hashing consistency, prevention of plain-text storage, credential rejection for invalid users, and strict role inheritance (Doctor, Nurse, Admin).
- Missing Data & Clinical Safety Rules: Confirmed accurate detection of missing required physiological features (e.g., None or Nan vitals) while safely handling optional laboratory parameters (Lactate, Creatinine).
- ML Inference & Cache Optimization: Verified deterministic output generation, correct caching of calculated risk levels (CRITICAL, HIGH, MODERATE, LOW), and graceful degradation to ERROR / INCOMPLETE statuses upon missing data or pipeline interruptions.

Automated Test Category	Focus Area	Executed Tests	Result
Authentication & RBAC	Password hashing, login validation, user deletion, role preservation.	8 Tests	Passed (100%)
Database & Persistence	Patient lifecycle, updates, notes, alert statuses, duplicate constraints.	7 Tests	Passed (100%)
Missing Data Detection	Vitals completeness, NaN handling, optional lab allowance.	5 Tests	Passed (100%)
ML Inference & Safety	Determinism, risk scoring, prediction caching, error handling.	5 Tests	Passed (100%)

6.8.2. Manual End-to-End User Journey Testing

Manual test cases were executed to validate UI flows, real-time response workflows, clinical data entry validations, and file export integrity.

Test ID	Test Scenario / Feature	Test Steps Executed	Expected Result	Actual Result / Status
---------	-------------------------	---------------------	-----------------	------------------------



TC-01	Role-Based Authentication & Demo Fill	1. Clicked the Doctor demo shortcut button. 2. Verified auto-filled credentials. 3. Clicked Login.	The system authenticates doctor user and loads the main Dashboard with authorized clinical permissions.	Passed, Logged in successfully as Dr. Sarah Cohen.
TC-02	Password Visibility Toggle	1. Entered password into field. 2. Clicked the eye icon toggle. 3. Clicked the toggle again.	Password switches from hidden to plain text and reverts back to hidden.	Passed, Text toggles as expected with matching icon changes.
TC-03	Department KPI Analytics & Live Refresh	1. Navigated to Department KPI. 2. Reviewed patient counts and risk stratification. 3. Clicked Refresh Live Metrics	The system accurately computes and displays aggregated ICU statistics and updates immediately.	Passed, Cards, averages, and stratification displayed correctly.
TC-04	Department CSV Data Export	1. Navigated to Department KPI. 2. Clicked Export Department to Excel / CSV. 3. Selected local file path and saved.	File save dialog opens: complete department dataset is exported as a formatted ".csv" file.	Passed, File saved and opened successfully with complete patient attributes.
TC-05	Overdue Urgency Banner Navigation	1. Identified a critical patient overdue for re-assessment. 2. Verified red banner on top of Dashboard. 3. Clicked View in Priority Queue.	Banner alerts staff and directs user to Priority Queue filtered automatically by CRITICAL.	Passed, Navigated directly to filtered critical patients view.
TC-06	Quick Review & Timer Reset in Priority Queue	1. Opened Priority Queue. 2. Located an overdue patient. 3. Clicked Mark Reviewed.	Re-assessment timer resets to active countdown: staff review is recorded in the database.	Passed, Status updated and timer reset green indicator displayed.



TC-07	Interactive Protocol Execution Checklist	1. Selected a patient and clicked Recommended Actions. 2. Checked clinical protocol items. 3. Clicked Acknowledge & Save Executed Protocol.	Executed protocol items are formatted with timestamp and appended to the patient's record.	Passed, Actions successfully recorded in patient chart.
TC-08	Patient Medical Report Export (.txt)	1. Opened Patient Summary. 2. Clicked Export Medical Report (.txt). 3. Chose save directory.	Comprehensive clinical report text file is generated with vitals, ML risk score, and notes.	Passed, .txt report saved and verified with correct structure.
TC-09	Role-Based Permission Enforcement	1. Logged in as Nurse Daniel Levi. 2. Attempted to edit the patient room and physician note.	System displays permission denial warnings for actions restricted to physicians only.	Passed, Authorization check blocked unauthorized edits.
TC-10	Vitals Input Validation & ML Recalculation	1. Opened Patient Details -> Edit Patient Vitals. 2. Entered an invalid heart rate (Min > Max). 3. Corrected values and saved.	System raises validation error for invalid range, and recalculates risk score upon valid input.	Passed, Error popup caught invalid entry; valid input updated risk score immediately.
TC-11	Theme Toggle (Dark / Light Mode)	1. Clicked Dark Mode on the sidebar. 2. Verified interface colors. 3. Clicked Light Mode.	UI smoothly switches color palette across all widgets, labels, and scrollable frames.	Passed, Interface rendered properly in both visual themes.

6.8.3 Testing Summary & Evaluation Verdict

- Automated Unit & Integration Tests: 25 / 25 Passed (100% Pass Rate)
- Manual End-to-End Scenarios: 11 / 11 Passed (100% Pass Rate)
- Final Verdict: The system achieves functional reliability, robust validation boundaries, zero regression across database transactions, and smooth clinical workflow support across both backend operations and desktop user interface components.

7. First paper

<https://arxiv.org/pdf/2404.16045>

Article objective & abstract

The paper "*Elictron: An LLM Agent-Based Simulation Framework for Design Requirements Elicitation*" addresses the fundamental challenge in systems engineering of uncovering hidden user needs and edge-case requirements early in the conceptual design phase. The objective of the research is to propose a framework that automates and enhances the requirement elicitation process by leveraging Large Language Models (= LLMs) to simulate diverse, realistic user personas (= ELU agents). By conducting automated multi-agent interactions and simulated user interviews, Elictron aims to identify latent needs that human engineering teams frequently overlook due to cognitive bias, limited diversity, or time constraints.

7.1 Proposed approach, main contribution, and key findings

The authors introduce **Elictron**, a multi-agent framework where specialized LLM agents enact diverse user personas, critics, and interviewers to systematically simulate requirements discovery. The primary contribution of this work is proving that automated multi-agent simulations can scale up requirement elicitation while maintaining high contextual accuracy. Key findings demonstrate that Elictron achieves a precision rate of up to **95%** (validated via blind expert evaluation) and an **F_1 - score of 0.95** when utilizing Chain-of-Thought prompting. Furthermore, empirical evaluations across smart home and autonomous vehicle case studies revealed that Elictron nearly doubled the discovery of latent user needs compared to traditional human engineering baselines.

7.2 Contribution to the project and how it is practical to application

In the context of our project, the Elictron framework provides a strong theoretical and methodological foundation for managing dynamic requirements and identifying hidden system constraints early in the design life cycle. By illustrating how LLM-based agent simulations can anticipate non-obvious user requirements (such as accessibility constraints, unexpected environmental conditions, or safety edge cases) before active development begins, the paper justifies the importance of early-stage requirement validation to prevent costly redesigns. We utilize the core principles of the paper to demonstrate how automated agent simulation can complement human analysis, reduce blind spots in system design, and ensure robust coverage of complex user scenarios.

8. Second paper

<https://arxiv.org/pdf/2406.11912>

Article objective & abstract

The paper "*AgileCoder: Dynamic Collaborative Agents for Software Development based on Agile Methodology*" addresses the disconnect between rigid, single-pass waterfall processes used in existing LLM-based frameworks and the reality of professional software engineering, where approximately 70% of teams utilize Agile. The objective of the research is to introduce **AgileCoder**, a multi-agent framework that maps Scrum roles , Product Manager, Scrum Master, Developer, and Tester , onto specialized agents to enable incremental, sprint-based software generation. Central to the system is a **Dynamic Code Graph Generator (DCGG)**,

which provides agents with token-efficient, up-to-date dependency context to support precise code generation, testing, and iterative bug-fixing as the codebase evolves.

8.1 Proposed approach, main contribution, and key findings

Iterative sprint cycle: AgileCoder maps professional Scrum workflows, Planning, Development, Testing, and Review, directly onto a multi-agent framework. Role-specific agents (Product Manager, Scrum Master, Developer, and Tester) manage a product backlog and execute incremental code refactoring under senior-level review. The Tester utilizes a topologically ordered plan to isolate and resolve bugs within the sprint boundary.

Dependency-aware context: The **Dynamic Code Graph Generator (DCGG)** serves as a technical constraint to manage LLM limitations. By tracking import-level relationships, the system retrieves only the necessary cross-file context. This prevents context-length overflow and reduces token overhead compared to loading a full repository snapshot.

Key findings: The framework established a new state-of-the-art for GPT-3.5 on HumanEval (**70.53%**) and MBPP (**80.92%**), outperforming existing baselines like MetaGPT and ChatDev by approximately 7.7%. On the more complex ProjectDev benchmark, it reached **57.79% executability** with zero non-functional outputs, nearly doubling the performance of legacy multi-agent systems.

Ablation results: The performance lift is modular. While incremental sprints and testing contributed gains, the dependency graph G was the dominant factor, increasing executability from 23.28% to 57.50% by eliminating context errors. This confirms that structured engineering context is as vital as the underlying model capacity.

8.2 Contribution to the project and how it is practical to application

The AgileCoder framework validates our core engineering philosophy: that a software system is a series of testable increments, not a monolithic event. Its feedback-driven loop, incorporating planning, execution, and senior-level review, directly mirrors our own *hypothesis-driven development process*. We moved from E001 to our final classifier through logged, manageable experiments with explicit failure criteria, rather than untracked bulk rewrites. Furthermore, the **Dynamic Code Graph Generator** illustrates the clinical necessity of mapping dependencies. In our system, the tight coupling between the feature-engineering pipeline, the LightGBM artifacts, and the CustomTkinter layer is an *explicit invariant*; AgileCoder's approach to dependency awareness provides a structural template for tracing how a modification in the backend propagation layer impacts the serving interface, effectively mitigating the risk of hidden-coupling bugs as the system scales.